

Tracing RLS: A Deep Dive into PostgreSQL's Row-Level Security Source Code

pg_policy.h ⇒ https://github.com/postgres/postgres/blob/master/src/include/catalog/pg_policy.h
rls.h ⇒ <https://github.com/postgres/postgres/blob/master/src/include/utils/rls.h>
rls.c ⇒ <https://github.com/postgres/postgres/blob/master/src/backend/utils/misc/rls.c>
rowsecurity.h ⇒ <https://github.com/postgres/postgres/blob/master/src/include/rewrite/rowsecurity.h>
rowsecurity.c ⇒ <https://github.com/postgres/postgres/blob/master/src/backend/rewrite/rowsecurity.c>
rewriteHandler.c ⇒ <https://github.com/postgres/postgres/blob/master/src/backend/rewrite/rewriteHandler.c>
fmgr.c ⇒ <https://github.com/postgres/postgres/blob/master/src/backend/utils/fmgr/fmgr.c>
miscinit.c ⇒ <https://github.com/postgres/postgres/blob/master/src/backend/utils/init/miscinit.c>
acl.h ⇒ <https://github.com/postgres/postgres/blob/master/src/include/utils/acl.h>
rowsecurity.sql ⇒ <https://github.com/postgres/postgres/blob/master/src/test/regress/sql/rowsecurity.sql>
rowsecurity.out ⇒ <https://github.com/postgres/postgres/blob/master/src/test/regress/expected/rowsecurity.out>

File No 1: pg_policy.h

https://github.com/postgres/postgres/blob/master/src/include/catalog/pg_policy.h

polrelid — *"which table does this policy belong to?"* When your query touches a relation, PostgreSQL looks up `pg_policy WHERE polrelid = <your table's oid>`. This is the lookup key. The index `pg_policy_polrelid_polname_index` makes this fast.

polcmd — *"which SQL command triggers this policy?"*

'r' = SELECT

'a' = INSERT

'w' = UPDATE

'd' = DELETE

'*' = ALL (every command)

During execution, only policies whose `polcmd` matches the current command (or is `''`) are picked up. A SELECT never sees an INSERT-only policy.

`polpermissive` — "how does this policy combine with others?"

This is the most important field for execution logic:

`true` = PERMISSIVE → combined with OR (any one passing = row visible)
`false` = RESTRICTIVE → combined with AND (all must pass = row visible)

Final qual built as: `(perm1 OR perm2 OR ...) AND restr1 AND restr2 AND ...`

`polroles` — "who does this policy apply to?" An array of role OIDs. Zero means `PUBLIC` (everyone). During execution, `check_role_for_policy()` checks if `current_user` is in this array. If not, the policy is completely skipped — not evaluated at all.

`polqual` — "the USING expression" The `pg_node_tree` (serialized expression tree) for the `USING (...)` clause. This is what gets injected as a `securityQual` into the RTE during the rewrite phase. It filters which existing rows are visible — applied on SELECT, UPDATE, DELETE.

`polwithcheck` — "the WITH CHECK expression" The `pg_node_tree` for `WITH CHECK (...)`. Applied on INSERT and UPDATE to validate rows being written. If this evaluates to false, you get `ERROR: new row violates row-level security policy`.

```
Query hits table
↓
lookup pg_policy WHERE polrelid = table_oid
↓
filter by polcmd (matches current command?)
↓
filter by polroles (current_user in array?)
↓
split into permissive / restrictive lists (polpermissive)
↓
polqual      → injected as USING filter (read path)
polwithcheck → injected as WCO check   (write path)
```

File No 2: rls.h

<https://github.com/postgres/postgres/blob/master/src/include/utils/rls.h>

The `row_security` GUC

```
extern PGDLLIMPORT bool row_security;
```

This is the `SET row_security = on/off` session variable. If a user sets `row_security = off` and they *don't* have bypass privilege, PostgreSQL **throws an error** rather than silently skipping RLS. This matters for your cycle detection — when you temporarily bypass RLS during a cycle, you are not touching this GUC. You bypass via `SetUserIdAndSecContext()` with the

`SECURITY_ROW_LEVEL_DISABLED` flag, which is a completely separate mechanism and doesn't interact with this variable.

`RLS_NONE_ENV` — the plan cache invalidation signal

`RLS_NONE` → RLS not on table, plan cache safe forever

`RLS_NONE_ENV` → RLS on table but bypassed RIGHT NOW, invalidate if env changes

`RLS_ENABLED` → RLS actively applying

`RLS_NONE_ENV` is the subtle one. It means *"RLS is defined on this table, but the current user is bypassing it (owner, superuser, BYPASSRLS)"*. The planner caches differently for this case — if the role changes, the plan must be replanned.

This is directly relevant to your feature: when you temporarily escalate during a cycle, the query is mid-execution so plan caching isn't a concern. But if you ever expose this as a declarable policy option (like `SECURITY DEFINER` on a policy), you'd need to return `RLS_NONE_ENV` rather than `RLS_NONE` so plans don't get incorrectly cached across role switches.

File No 3: `rls.c` (Here we check whether RLS applies to the query or not?)

<https://github.com/postgres/postgres/blob/master/src/backend/utils/misc/rls.c>

`check_enable_rls()` => !important This function decides **whether Row-Level Security (RLS) should be applied for a given table and user.**

```
int check_enable_rls(Oid relid, Oid checkAsUser, bool noError)
{
    Oid          user_id = OidIsValid(checkAsUser) ? checkAsUser : Get
    UserId();
    HeapTuple    tuple;
    Form_pg_class classform;
    bool         relrowsecurity;
    bool         relforcerowsecurity;
    bool         amowner;

    if (relid < (Oid) FirstNormalObjectId)
        return RLS_NONE;

    tuple = SearchSysCache1(RELOID, ObjectIdGetDatum(relid));
    if (!HeapTupleIsValid(tuple))
        return RLS_NONE;
    classform = (Form_pg_class) GETSTRUCT(tuple);
```

```

relrowsecurity = classform->relrowsecurity;
relforcerowsecurity = classform->relforcerowsecurity;

ReleaseSysCache(tuple);

if (!relrowsecurity)
    return RLS_NONE;

if (has_bypassrls_privilege(user_id))
    return RLS_NONE_ENV;

amowner = object_ownercheck(RelationRelationId, relid, user_id);
if (amowner)
{
    if (!relforcerowsecurity || InNoForceRLSOperation())
        return RLS_NONE_ENV;
}

if (!row_security && !noError)
    ereport(ERROR,
            (errcode(ERRCODE_INSUFFICIENT_PRIVILEGE),
             errmsg("query would be affected by row-level security policy for table \"%s\"",
                    get_rel_name(relid)),
             amowner ? errhint("To disable the policy for the table's owner, use ALTER TABLE NO FORCE ROW LEVEL SECURITY.") : 0));

return RLS_ENABLED;
}

```

`checkAsUser` parameter

```

Oid user_id = OidIsValid(checkAsUser) ? checkAsUser : GetUserId();

```

When views run RLS checks, they pass the view owner's OID here instead of the session user. This is the same mechanism you'll use — when you temporarily escalate during a cycle, you're effectively calling the downstream check as a different user. Understanding this parameter is key because your bypass needs to short-circuit this function entirely, not just pass a different OID.

!Important

If caller explicitly passes a user → use that. Otherwise: `GetUserId()` uses current active user.
⇒ This is why in SECURITY DEFINER we are passing function owner credentials.

`InNoForceRLSOperation()` This is a context flag set during referential integrity checks (foreign key triggers). When Postgres checks FK constraints, it sets this flag so that even if a table has `FORCE ROW LEVEL SECURITY`, the owner can still see all rows needed for the integrity check. This is a precedent for your feature — Postgres already has a mechanism to say "we're in a special internal context, suppress RLS". Your cycle detection flag will work the same way.

The `noError` path

```
if (!row_security && !noError)
    ereport(ERROR, ...)
```

When `noError = true`, even if `row_security = off`, it returns `RLS_ENABLED` silently instead of erroring. This is used in error-reporting paths where Postgres wants to decide "should I show this row in the error message?" without triggering another error. Your cycle detection should pass `noError = true` when calling this during escalation to avoid double-error situations.

```
reloid < FirstNormalObjectId? → RLS_NONE    (system tables never get
RLS)
    ↓
relrowsecurity = false?      → RLS_NONE    (table has no policies at
all)
    ↓
has_bypassrls_privilege()? → RLS_NONE_ENV (superuser / BYPASSRLS
role)
    ↓
amowner = true?
    └─ FORCE RLS not set?    → RLS_NONE_ENV (owner bypasses normal
ly)
    └─ FORCE RLS set
        AND NOT InNoForceRLSOperation() → falls through to RLS_ENABL
ED
    ↓
row_security GUC = false?   → ereport(ERROR) (user opted into har
d error)
```

↓

→ RLS_ENABLED

File No 4: rowsecurity.h

<https://github.com/postgres/postgres/blob/master/src/include/rewrite/rowsecurity.h>

`RowSecurityPolicy` — one policy in memory

This is the in-memory representation of one `pg_policy` row after it's been loaded from syscache. Compare it directly to what you read in `pg_policy.h`:

```
policy_name    ← polname      (identity only, not used in execution)
polcmd         ← polcmd       ('r', 'a', 'w', 'd', '*')
roles         ← polroles     (ArrayType*, checked via check_role_for_policy)
permissive     ← polpermissive (true=OR, false=AND)
qual          ← polqual      (USING expression, already parsed into Expr*)
with_check_qual ← polwithcheck (WITH CHECK expression, already parsed into Expr*)
hassublinks    ← (computed)   whether qual or with_check_qual contains a subquery
```

!Important:

`hassublinks` is the field most relevant to your cycle detection — if `hassublinks = true`, that policy's `qual` contains a subquery, and that subquery might reference the same table, which is exactly how a cycle forms. This is your first detection signal.

`RowSecurityDesc` — all policies for one relation

```
typedef struct RowSecurityDesc {
    MemoryContext rscxt; /* owns the memory for all policies below */
    List          *policies; /* list of RowSecurityPolicy* */
} RowSecurityDesc;
```

This lives in the `relcache` — cached per-relation. When `get_row_security_policies()` is called, it reads from this cached descriptor rather than hitting `pg_policy` every time. This means policy

changes require a relcache invalidation to take effect (which `rewriteDefine.c` triggers via `CacheInvalidateRelcache()`).

```
row_security_policy_hook_permissive / row_security_policy_hook_restrictive
```

```
typedef List *(*row_security_policy_hook_type)(CmdType cmdtype, Relation relation);

extern row_security_policy_hook_type row_security_policy_hook_permissive;
extern row_security_policy_hook_type row_security_policy_hook_restrictive;
```

These are extension hooks. If a shared library sets these function pointers, Postgres calls them during policy evaluation and merges the returned policy list with the ones from `pg_policy`. This is how extensions like Citus or `pg_separation` inject their own RLS policies without modifying core code.

For your cycle detection — these hooks are called inside `get_row_security_policies()`, so any cycle involving extension-injected policies would also be caught by your depth counter at the same interception point.

`get_row_security_policies()` — the main entry point

```
void get_row_security_policies(
    Query *root, /* the full query being rewritten */
    RangeTblEntry *rte, /* the specific table RTE being processed */
    int rt_index, /* index of this RTE in the query's rtable */
    List **securityQuals, /* OUT: USING quals to inject */
    List **withCheckOptions, /* OUT: WITH CHECK quals to inject */
    bool *hasRowSecurity, /* OUT: did we apply any RLS? */
    bool *hasSubLinks /* OUT: do any quals have subqueries? */
);
```

It takes a single RTE, evaluates all matching policies for it, and hands back two output lists — `securityQuals` (for reads) and `withCheckOptions` (for writes). The caller (`fireIRrules()` in

`rewriteHandler.c`) staples these onto the RTE.

!Important

The `hasSubLinks` output is what `fireRIRrules()` uses to decide whether it needs to recursively call `fireRIRonSubLink()` – which is the recursive descent that can cause a cycle. This is the exact callsite where your depth counter needs to live.

File No 5: rowsecurity.c (1k line of code 🤔)(Its only job is to answer one question: "For this table and this user, what WHERE clause should I secretly add to this query?")

<https://github.com/postgres/postgres/blob/master/src/backend/rewrite/rowsecurity.c>

This file (`rowsecurity.c`) is that bouncer's brain. Every time a query touches a table, this code runs and asks *"what rows is this person allowed to see?"*

`get_row_security_policies()` — the entry point

This is called by `fireRIRrules()` for every RTE(**Range Table Entry**) in the query. The flow inside it:

```
check_relkind → only RELATION and PARTITIONED_TABLE get RLS
↓
get_checkAsUser from perminfo (view owner override)
↓
check_enable_rls() → RLS_NONE / RLS_NONE_ENV / RLS_ENABLED
↓
if RLS_NONE_ENV → set hasRowSecurity=true, return (plan cache signal)
↓
table_open() → get Relation object (has rd_rsdsc with cached policies)
↓
decide_commandType → if this RTE is not the target relation, force CMD_SELECT
↓
call get_policies_for_relation() + add_security_qual() / add_with_check_options()
↓
table_close()
↓
setRuleCheckAsUser() → propagate checkAsUser into any subqueries in quals
↓
set hasRowSecurity = true
```

Simple terms:

```

Is this even a real table? → No? Leave immediately
↓
Does this table have RLS? → No? Leave immediately
↓
Is this person a superuser? → Yes? Let them through, no checks
↓
Ok RLS applies – figure out WHICH policies to use
↓
Build the filter and hand it back to the query

```

`get_policies_for_relation()` — policy filtering

This function answers: **"Which policies actually apply to this person right now?"**

Reads from `relation->rd_rsdsc->policies` — this is the relcache, not a live syscache hit. Two filters applied to each policy:

Filter 1 — command match:

```

policy->polcmd == '*' → always matches (ALL policies)
policy->polcmd == 'r' → CMD_SELECT only
policy->polcmd == 'a' → CMD_INSERT only
policy->polcmd == 'w' → CMD_UPDATE only
policy->polcmd == 'd' → CMD_DELETE only
CMD_MERGE           → never matches directly (derived from others)

```

Filter 2 — role match via `check_role_for_policy()`:

```

roles[0] == ACL_ID_PUBLIC → true immediately (PUBLIC policy)
otherwise → has_privs_of_role(user_id, roles[i]) ← role inheritance aware

```

If the policy is for `PUBLIC` → everyone passes. Otherwise it checks if the current user is in the allowed roles list — and it's smart enough to check role inheritance too (if you're a member of `managers` group, you get `managers` policies).

After filtering, policies are split into two lists:

```

permissive_policies → policy->permissive == true
restrictive_policies → policy->permissive == false

```

`add_security_qual()` — builds the USING filter

This is where the actual SQL filter condition gets built. Think of it like building a `WHERE` clause automatically.

This is where the OR/AND combination logic lives. The final qual injected into `securityQuals`:

```
No "allow" policies found at all?
→ inject WHERE false  (nobody sees anything – default deny)

"Allow" policies found?
→ combine them with OR  (pass any one = row is visible)
→ combine "must pass" policies with AND  (must pass ALL of them too)
```

`add_with_check_options()` — builds the write-path check

Same idea as above but for **INSERT** and **UPDATE** instead of SELECT.

```
SELECT → uses securityQuals  (filters what you READ)
INSERT → uses WithCheckOptions (validates what you WRITE)
UPDATE → uses BOTH
```

One subtle thing — there are two expressions per policy:

```
USING (...) → controls visibility (read path)
WITH CHECK (...) → controls what you can write (write path)
```

`check_role_for_policy()` — role matching

```
if (roles[0] == ACL_ID_PUBLIC)
    return true;  // PUBLIC policy, skip loop entirely

for each role in policy_roles:
    if has_privs_of_role(user_id, roles[i])
        return true;
```

The simplest function. Just answers: *"Does this policy apply to this user?"*

The key detail — `has_privs_of_role()` understands role inheritance. If policy says `FOR ROLE managers` and you are a member of `managers`, you pass — even though your personal role name isn't `managers`.

What this means `for` cycle detection

The cycle entry point is here in `get_row_security_policies()`:

```
rel = table_open(rte->relid, NoLock);
// ...
get_policies_for_relation(rel, commandType, user_id,
                        &permissive_policies, &restrictive_policies);
add_security_qual(..., hasSubLinks); // ← hasSubLinks set to true here
```

When `hasSubLinks = true` is returned, `fireRIRrules()` in `rewriteHandler.c` will call `fireRIRonSubLink()` to process subqueries inside those quals – and `if` a subquery references the same table, `get_row_security_policies()` gets called again `for` the same `relid`. That is the cycle.

Your depth counter goes at the very top of `get_row_security_policies()`, keyed on `rte->relid`. If you enter this function `for` a `relid` that's already being processed higher up the call stack, that's your cycle signal.

File 6: `rewriteHandler.c` — This is where Postgres silently injects the RLS `USING(...)` filter into every query that touches a table with policies.

<https://github.com/postgres/postgres/blob/master/src/backend/rewrite/rewriteHandler.c>

`rewriteHandler.c` is the main query rewriter — every query passes through its entry point `QueryRewrite()` before the planner sees it. It first applies non-SELECT rules like INSERT/UPDATE/DELETE rules via `RewriteQuery()`, then calls `fireRIRrules()` which does two things relevant to RLS: it expands views by replacing view RTEs with their underlying subqueries, and then in a separate loop it calls `get_row_security_policies()` from `rowsecurity.c` for every table RTE in the query — taking back the security quals and silently injecting them into the query tree via `rte->securityQuals`. If any of those injected policy quals contain subqueries (i.e. `hasSubLinks = true`), it recursively walks into those subqueries via `fireRIRonSubLink()` and applies RLS there too — and it is exactly at this recursive step where a

cycle can form, which currently Postgres handles by throwing a hard error via `list_member_oid(activeRIRs, relid)` check, and which your feature needs to replace with a graceful bypass instead.

File 7: `miscinit.c`

Stores and manages the current user identity (`CurrentUserId`) and security context flags (`SecurityRestrictionContext`) for a running Postgres session. Provides `GetUserIdAndSecContext()` and `SetUserIdAndSecContext()` which are the two functions you call to save, switch, and restore the security context during your RLS cycle detection bypass — exactly the same way `SECURITY DEFINER` uses them to temporarily escalate privileges.

File 8: `acl.h`

Defines PostgreSQL's access control and privilege system. For RLS, only one function matters: `has_bypassrls_privilege(roleid)` which is called inside `check_enable_rls()` to check if the current user should skip all RLS policies entirely — understanding this confirms that your cycle detection bypass should use `SECURITY_NOFORCE_RLS` flag rather than faking a bypass privilege.

File 9: `rowsecurity.sql`

<https://github.com/postgres/postgres/blob/master/src/test/regress/sql/rowsecurity.sql>

```
----- RECURSION -----

-- Simple recursion
CREATE TABLE rec1 (x integer, y integer);
CREATE POLICY r1 ON rec1 USING (x = (SELECT r.x FROM rec1 r WHERE y =
r.y));
ALTER TABLE rec1 ENABLE ROW LEVEL SECURITY;
SET SESSION AUTHORIZATION regress_rls_bob;
SELECT * FROM rec1; -- fail, direct recursion

-- Mutual recursion
CREATE TABLE rec2 (a integer, b integer);
ALTER POLICY r1 ON rec1 USING (x = (SELECT a FROM rec2 WHERE b = y));
CREATE POLICY r2 ON rec2 USING (a = (SELECT x FROM rec1 WHERE y =
b));
SELECT * FROM rec1; -- fail, mutual recursion
```

```
-- Mutual recursion via views
-- Mutual recursion via security barrier views
```

These are the exact test cases that currently throw errors — and these are precisely the cases your feature needs to handle gracefully instead. Your new regression test file will be a modified version of this section where instead of `-- fail, direct recursion` the expected output is rows returned successfully with RLS bypassed for the inner call.

`test cases for free.`

This is `rowsecurity.out` — the expected output file that the regression test framework diffs against.

```
SELECT * FROM rec1; -- fail, direct recursion
ERROR:  infinite recursion detected in policy for relation "rec1"

SELECT * FROM rec1;    -- fail, mutual recursion
ERROR:  infinite recursion detected in policy for relation "rec1"

SELECT * FROM rec1;    -- fail, mutual recursion via views
ERROR:  infinite recursion detected in policy for relation "rec1"

SELECT * FROM rec1;    -- fail, mutual recursion via s.b. views
ERROR:  infinite recursion detected in policy for relation "rec1"
```

After implementation, these same queries should either return rows (if the table has data) or return empty results — but not error.